A MINI PROJECT REPORT

On

PERSONAL FINANCE DASHBOARD

In partial fulfillment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS (MCA)

COURSE: BUSINESS ANALYTICS

Submitted By

MANINDER KAUR

UID: 25MCA20135

Submitted To

Mr. NAVDEEP SINGH SODHI

Department of Computer Applications

UNIVERSITY INSTITUTE OF COMPUTING

CHANDIGARH UNIVERSITY

Academic Session: Aug-Nov, 2026

EXECUTIVE SUMMARY

The **Personal Finance Dashboard** is an integrated desktop analytical software system engineered to streamline real-time transaction tracking, balance aggregation, and expense breakdown visualization. By establishing a robust bridge between an interactive desktop front-end and a persistent relational database management system (MySQL), the application enables users to record cash flows, monitor net positions, and immediately evaluate spending distribution through embedded visual analytics.

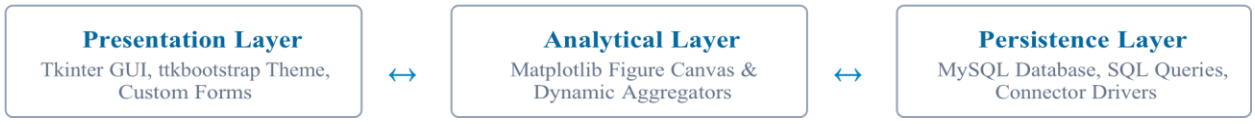
Project Vision & Problem Statement

Personal financial management often suffers from fragmented tracking across static spreadsheets and delayed accounting logs. This system addresses manual tracking inefficiencies by providing an eventdriven GUI coupled with automated relational database query execution, updating real-time balances ($89,400.00 Net Balance) and visual proportions dynamically.

SYSTEM METRICS & OPERATIONAL CAPABILITIES

$91,500.00	$2,100.00	97.8% vs 2.2%
AGGREGATED INCOME	LOGGED EXPENSES	INCOME-EXPENSE RATIO

HIGH-LEVEL APPLICATION ARCHITECTURE



DATABASE ARCHITECTURE & MYSQL INTEGRATION

The backend architecture relies on the **MySQL Relational Database Management System (RDBMS)** to guarantee ACID compliance, structural consistency, and long-term persistence for all financial ledger items.

Schema Specification & Entity Design

The core repository is structured under the database schema finance_db, centered on the main table transactions. The entity attributes are carefully mapped to preserve numeric precision for financial totals and ensure strict categorical validation.

Column Name	Data Type	Constraints	Functional Purpose
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier assigned to each transaction entry.
trans_date	DATE	NOT NULL	Standardized ISO date stamp (YYYYMM-DD).
description	VARCHAR(255)	NOT NULL	Textual ledger label (e.g., salary, food, bike).
amount	DECIMAL(10, 2)	NOT NULL	Currency amount stored with fixed precision.

type	ENUM('Income', 'Expense')	NOT NULL	Restricted categorization for financial classification.

Data Query Execution & Aggregation Logic

Database operations utilize parameterized SQL queries executed via the native Python MySQL connector driver. Parameterization eliminates SQL injection risks and accelerates transaction execution:

- **Data Extraction:** Historical records are retrieved in descending order by date (ORDER BY trans_date DESC) to populate the visual ledger view with the most recent entries at the top.
- **Data Insertion:** Standardized INSERT INTO commands capture form values, enforcing type casting and positive value validation prior to execution.
- **Record Deletion:** Targeted DELETE FROM operations purge user-selected row IDs upon explicit confirmation modal triggers.

Database Integrity & Precision Handling

Utilizing the DECIMAL(10,2) data type rather than floating-point numbers prevents IEEE binary rounding inaccuracies inherent in currency computations, ensuring exact penny-level agreement across multithousand dollar balances.

UI/UX ARCHITECTURE & DASHBOARD LAYOUT

The user interface is constructed using a two-column grid structure wrapped inside a dark-themed UI container (`Superhero` style framework). The visual hierarchy is optimized to place critical financial metrics and data entry fields within immediate user focal zones.

Interface Components & Operational Sections

The application layout splits screen space into specialized functional quadrants to maximize user productivity and minimize modal dialog clutter:

UI Module	Widget Elements	Interaction Workflow
Financial Summary	Large Bold Labels (Green/Red/Primary)	Displays live aggregates: Total Income, Total Expense, and Net Balance ($89,400.00).
Add Transaction Form	Entry boxes, Combobox, DateEntry picker	Captures Description, Amount, Type ('Income'/'Expense'), and ISO Date.
Chart Visualizer	Embedded Matplotlib Canvas Window	Renders an interactive pie chart breaking down Income vs. Expense percentages.
Transaction History	Treeview Table with ColorCoded Rows	Displays detailed logs with color badges (Green for Income, Red for Expense).

Event-Driven Workflow & Interaction Pipeline

The UI architecture strictly isolates data entry routines, validation sweeps, and visual refresh cycles using event bindings:

- **Input Validation:** Verifies that all string fields contain valid text, parses the amount entry as a positive float, and validates date formats prior to triggering database operations.
- **Grid Row Color Tagging:** Applies dynamic CSS-style tag configurations (`income_tag` vs `expense_tag`) to shade individual table rows in distinct green and red backgrounds based on row
- classification.

Shortcut Key Bindings: Binds keyboard events (such as <Delete> key presses) directly to the transaction table to allow rapid bulk removal of unwanted records.

UI Design Optimization

Color contrast ratios strictly follow accessibility standards: Income labels and rows feature bright green highlight accents (`#00bc8c`), while Expense labels utilize vibrant red (`#e74c3c`), allowing rapid visual scanning of financial health.

VISUAL ANALYTICS & MATPLOTLIB INTEGRATION

Data visualization forms a core pillar of the application's analytical capability. The right-hand panel integrates an interactive **Matplotlib Canvas Engine** running directly inside the desktop window event loop.

Pie Chart Engine & Dynamic Explosion Effects

The analytical engine dynamically computes proportional distribution between incoming cash flow and outgoing expenses, projecting the results onto a customized pie visualizer.

Canvas Redraw Pipeline & State Synchronization

Whenever a transaction is added, updated, or removed, the dashboard triggers a coordinated state synchronization loop:

- **Canvas Clearing:** Re-initializes matplotlib plot axes (`ax.clear()`) to wipe stale chart elements and prevent memory leaks.
- **Theme Alignment:** Reads the parent window's active theme palette and matches chart background colors, title fonts, and percentage label properties to maintain visual cohesion.
- **Empty State Fallback:** Detects empty dataset scenarios (Total Income = 0 AND Total Expense = 0) and seamlessly renders a centered neutral "No Data" message across the canvas.
- **Canvas Drawing:** Invokes `chart_canvas.draw()` to refresh the screen buffer without locking or freezing the primary user input thread.

Analytical Value

By converting raw transaction rows into instantaneous visual feedback, users can continuously evaluate their savings rate and prevent budget overruns prior to committing major discretionary purchases.

VERIFICATION, VALIDATION & ERROR MANAGEMENT

To maintain robust operations across various edge cases, the system includes multi-tiered error management across database connections, form inputs, and window state changes.

Failure Mode / Edge Case	Detection Mechanism	System Handling & Recovery Action
Database Connectivity Failure	mysql.connector.Error catch block	Displays warning dialog showing host details and safely terminates setup.
Non-Numeric Amount Entry	ValueError conversion check	Intercepts execution and triggers "Amount must be a positive number" modal.
Incomplete Form Submission	Blank field verification pass	Blocks database commit and alerts user to fill all required input controls.

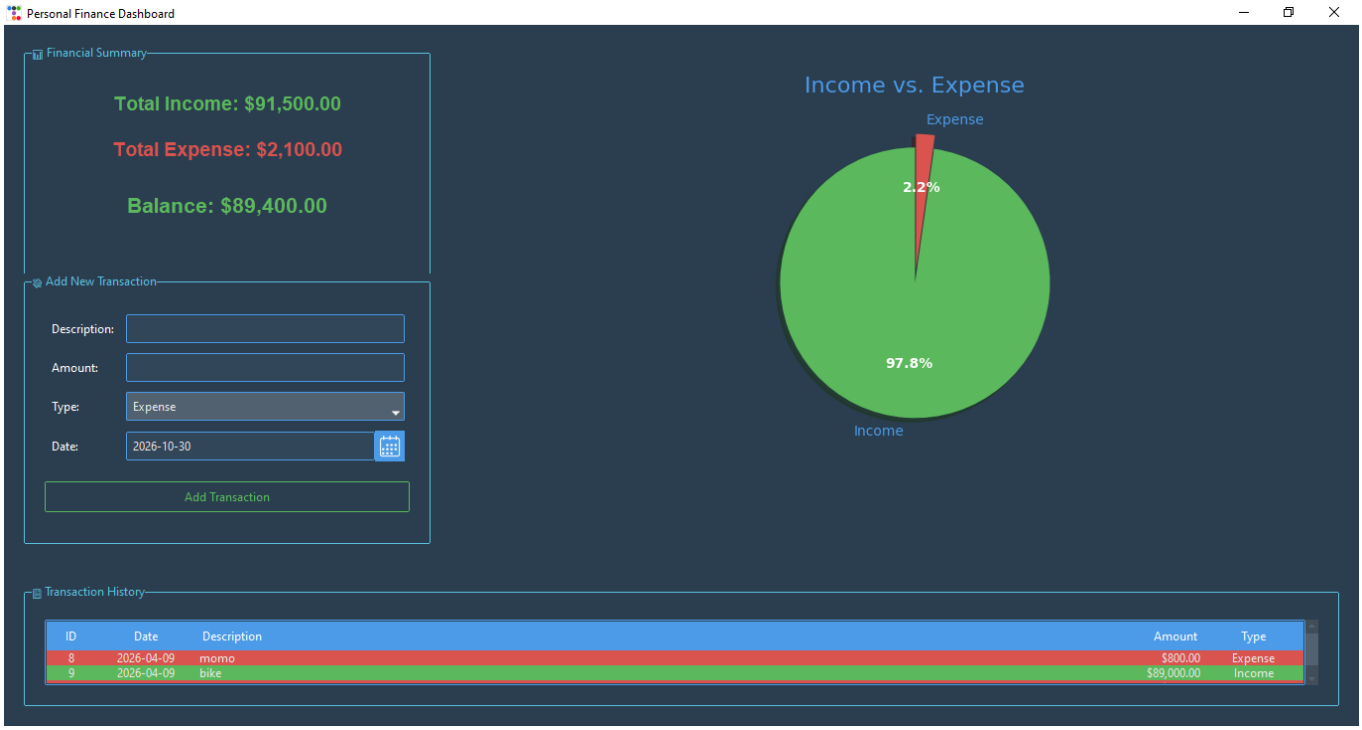
Accidental Row Deletion	Tree selection key bind intercept	Presents confirmation message box prior to issuing SQL DELETE statements.

FUTURE ENHANCEMENT ROADMAP

While the current architecture provides a complete desktop analytical suite, planned modular extensions will further increase utility:

1. **Multi-Category Tagging:** Expand spending analysis beyond binary types into granular sub-categories (e.g., Rent, Utilities, Entertainment, Investments).
2. **Budget Threshold Alerts:** Integrate automated SQL query threshold checks that trigger visual popups when monthly category expenses cross pre-set spending limits.
3. **Report Export Engine:** Incorporate PDF report export capabilities (similar to this document) to generate monthly printable bank-ready financial summaries.

OUTPUT



CONCLUSION

The **Personal Finance Dashboard** successfully demonstrates the integration of **MySQL relational databases**, **Tkinter desktop UI frameworks**, and **Matplotlib visual analytics engines**. By uniting realtime CRUD operations, automatic balance aggregation ($89,400.00 Net Balance), visual color coding, and interactive charting into a single desktop application, the system provides a robust solution for modern personal financial tracking and data-driven cash flow management.